

CloudAlloc

Predictive CPU and Memory Allocation for Cloud Workloads

30% milestone - four-person explanation and live demonstration guide

PRESENTATION	CURRENT BUILD	DEMO READY
4 speakers / 18-22 min	Foundation milestone only	SQLite fixture workflow

Use this document as speaking notes, a viva reference, and the exact runbook for demonstrating what is implemented today.

Important: generated fixture results prove software behavior. They are not empirical findings about real Google workloads.



TEAM OVERVIEW

Presentation map and current scope

Speaker	Time	Primary responsibility	Handoff line
Person 1	4-5 min	Problem, research gap, objectives, dataset, and methodology	Now we will show how that research design becomes a system.
Person 2	4-5 min	Architecture, preprocessing, database, manifest, and reproducibility	With clean data stored, the next question is what the current analysis can prove.
Person 3	4-5 min	EDA, persistence baseline, API, tests, results, and code walkthrough	The final speaker will run the complete current workflow live.
Person 4	5-7 min	Live setup, commands, output interpretation, limitations, and next 70%	That completes the implemented 30% foundation.

What exists now

- Google trace streaming preprocessor and normalized Parquet contract.
- Checksum manifest and chronological train/validation/test labels.
- PostgreSQL-ready relational schema with SQLite demonstration mode.
- Idempotent ingestion, training-only EDA, persistence baseline.
- Health and workload-summary endpoints plus 13 automated tests.

What does not exist yet

- Ridge, random forest, and gradient-boosting experiments.
- Static/reactive/predictive/oracle allocation simulator.
- Waste, deficit, utilization, churn, and confidence intervals.
- Recommendation endpoint and final analytics dashboard.
- Real-trace empirical conclusions.

One-sentence project description

CloudAlloc is a reproducible research prototype that prepares cloud workload traces and establishes a leakage-safe baseline before later converting CPU and memory forecasts into allocation recommendations.

SPEAKER SECTION 1 OF 4

Person 1 - problem, gap, and purpose

PERSON 1 - RESEARCH LEAD

Suggested opening

Say this first

Cloud workloads do not use a constant amount of CPU or memory. If resources are fixed too high, capacity is wasted. If they are fixed too low, performance and reliability suffer. Reactive scaling also acts only after demand changes.

Points to explain

- Static allocation reserves capacity for a demand level that may not occur.
- Reactive threshold methods have a one-step delay: they observe pressure first and respond afterward.
- Forecasting alone is not enough. A model may have low average error but still under-predict peaks that matter for allocation.
- The research gap is the missing link between prediction quality and operational outcomes on the same public workload trace.
- The project therefore evaluates prediction and, in later milestones, allocation waste and shortage together.

Research questions

#	Question
1	How accurately can next-interval maximum CPU and memory demand be predicted?
2	Can predictive allocation reduce waste relative to static requests?
3	Can it reduce under-provisioning relative to delayed reactive scaling?
4	Does the most accurate model also produce the best allocation outcome?

Do not overclaim

- At 30%, only Question 1 has a persistence baseline implementation, and only on the validation split.
- Questions 2-4 require the allocation simulator and advanced models planned for later milestones.

Person 1 - dataset and experimental method

Why Google Cluster Trace 2019?

- Its instance-usage records include average and maximum CPU and memory signals.
- Its instance-event records include requested CPU and memory plus scheduling metadata.
- The same task intervals can therefore support forecasting and later allocation comparisons.
- The full trace is roughly 2.4 TiB compressed, so the design uses a deterministic laptop-sized sample rather than the full dataset.

Design choice	Decision	Reason
Cell	Google cell a	Fixed scope and reproducibility.
Window	One contiguous seven-day period	Preserves temporal order and near-term patterns.
Cohort	Up to 1,000 long-running tasks	Feasible on an 8-16 GB laptop.
Coverage	At least 80% expected intervals	Avoids highly fragmented sequences.
Horizon	Next five-minute interval	Matches the trace sampling interval.
Split	60% train / 20% validation / 20% test	Prevents random time leakage.
Units	Normalized capacity	Public values are not physical vCPU or GB.

Key methodological safeguard

An event is joined only when its event time is no later than the usage sample. One missing interval can be filled only from the previous observation. Forecast pairs must be exactly five minutes apart.

Handoff to Person 2

The research design tells us what data is valid. Person 2 will now explain how the code turns raw trace records into a reproducible database.

SPEAKER SECTION 2 OF 4

Person 2 - architecture and data pipeline

PERSON 2 - DATA AND DATABASE

Explain the pipeline from left to right

Stage	Input	Responsibility	Output
1. Read	Official JSON or JSON.GZ shards	Stream records instead of loading the whole trace.	Usage/event dictionaries
2. Normalize	Nested Google fields	Rename and type CPU, memory, timestamps, IDs, request metadata.	Standard tabular rows
3. Select	Candidate task rows	Stable SHA-256 hash, time window, coverage threshold, task cap.	Reproducible cohort
4. Join	Usage and events	Attach only the latest request known at each sample time.	Leakage-safe records
5. Validate	Normalized frame	Reject missing required fields, negative values, bad times, duplicates.	Clean frame
6. Package	Clean frame	Write Zstandard Parquet and checksum manifest.	Versioned dataset
7. Ingest	Manifest plus Parquet	Verify checksum and insert only records not already present.	Relational database

Why a manifest matters

It records the schema version, dataset version, source, cell, task rule, row/task counts, trace range, units, file path, and SHA-256. If the Parquet file changes, ingestion rejects it.

Core code to mention

- cloudalloc/preprocessing.py owns streaming, validation, splits, Parquet, and manifest logic.
- cloudalloc/ingestion.py maps trace identities to database task IDs and performs idempotent inserts.
- cloudalloc/sample_data.py generates the deterministic four-task fixture used for a safe demo.

SPEAKER SECTION 2 OF 4

Person 2 - relational model and reproducibility

Database entities

Table	Purpose now	Purpose later
tasks	One row per trace task identity and scheduling metadata.	Groups histories and bootstrap units.
workload_samples	Timestamped requests and CPU/memory usage with split label.	Feature and allocation inputs.
experiments	Registers the persistence baseline and artifact path.	Registers each model/version/configuration.
predictions	Stores validation persistence predictions and actual targets.	Stores all model forecasts.
evaluation_metrics	Stores MAE, RMSE, sMAPE, and R2.	Stores policy metrics and confidence intervals.
allocation_decisions	Scaffolded and intentionally empty.	Stores static, reactive, predictive, and oracle decisions.

Integrity controls

- Unique task trace identity.
- Unique task plus timestamp sample.
- Unique experiment prediction target.
- Timestamp and lookup indexes.
- Alembic migration at revision 0001.

Reproducibility controls

- Stable task hash and fixed random behavior.
- Content checksum and schema version.
- Chronological split labels stored with samples.
- Raw data and generated artifacts excluded from Git.
- Synthetic fixture clearly labelled non-research.

Idempotency demonstration

The first seed creates 4 tasks and inserts 143 samples. Running the same command again creates 0 tasks and inserts 0 samples. This proves that duplicate ingestion is safely ignored.

Handoff to Person 3

Once the data is validated and stored, Person 3 can explain what analysis, baseline forecasting, APIs, and tests are working today.

Person 3 - EDA and persistence baseline

PERSON 3 - ANALYSIS AND BACKEND

Exploratory data analysis

- EDA defaults to the training split, so the test period is not inspected during model development.
- It writes eda_summary.json for reproducible statistics and eda.html for interactive Plotly charts.
- It covers distributions, missingness, correlations, temporal CPU/memory behavior, and request-versus-usage gaps.

Persistence forecast

Definition

For task i , the predicted maximum CPU or memory at $t+1$ equals the observed maximum at t . This simple method is important because stable workloads can make it difficult for more complex models to show genuine improvement.

Metric	What it answers	Direction
MAE	What is the typical absolute prediction error?	Lower is better
RMSE	How strongly are larger errors penalized?	Lower is better
sMAPE	How large is relative error with zero-safe handling?	Lower is better
R2	How much target variation is explained?	Higher is better

Current fixture output - demonstration only

Resource	Validation MAE	Validation RMSE	Validation sMAPE	Validation R2
CPU	0.00985	0.01169	0.04771	0.98317
Memory	0.00400	0.00478	0.01573	0.99014

These values describe generated periodic data. Do not use them to argue that the model performs well on Google workloads.

SPEAKER SECTION 3 OF 4

Person 3 - API, tests, and code responsibilities

Current web interface

Route	Responsibility	What to point out
GET /	Shows the 30% milestone landing page.	It openly labels later features as future work.
GET /api/health	Checks application and database availability.	Returns status, database state, and milestone label.
GET /api/workloads/summary	Aggregates task/sample counts, trace range, requests, and usage.	Values are labelled normalized capacity.
GET /api/workloads/summary?start=...&end;=...	Applies trace-time filters.	Invalid or reversed ranges return JSON 400 errors.

Automated evidence

- 13 tests pass in the current environment.
- Preprocessing tests cover stable hashing, negative-value rejection, gap filling, chronological splits, checksum tampering, and prior-event joins.
- Integration tests prove idempotent ingestion and database-to-JSON API behavior.
- Baseline tests prove no prediction crosses a gap, perfect metrics are handled, and experiments/predictions/metrics are persisted.
- CLI tests run the seed and EDA workflows end to end.

Best code explanation order

Start at `wsgi.py` and `create_app()`, then follow CLI commands into preprocessing/ingestion, models, analysis/baseline, and finally API responses. This mirrors the real data flow and is easier to understand than explaining files alphabetically.

Handoff to Person 4

The implementation is testable and produces stored results. Person 4 will now reproduce that full current-state workflow live.

SPEAKER SECTION 4 OF 4 - DEDICATED RUN SECTION

Person 4 - live demo: setup and database

PERSON 4 - DEMO AND ROADMAP

Recommended demo mode

Use SQLite and the deterministic fixture. It avoids Docker, network, PostgreSQL credentials, and the multi-terabyte trace while exercising the same application, models, tables, CLI, EDA, baseline, and API paths.

Before the presentation

- Open PowerShell in D:\projects\dbms.
- Confirm Python 3.11 or newer is available.
- Close any process already using port 5000.
- Keep reports/generated/eda.html ready as a backup if live Plotly generation is slow.
- Increase terminal font size and browser zoom for the audience.

Step 1 - install once

```
cd D:\projects\dbms
python -m pip install -e ".[dev]"
```

Step 2 - select the app and a local SQLite database

```
$env:FLASK_APP = "wsgi.py"
$env:DATABASE_URL = "sqlite:///cloudalloc-demo.sqlite3"
```

Step 3 - apply the relational migration

```
python -m flask db upgrade
python -m flask db current
```

Expected explanation

- Revision 0001 creates all planned tables and constraints. SQLite is only the demo database; PostgreSQL is the intended deployment database.

DEDICATED CURRENT-STATE RUN SECTION

Person 4 - live demo: generate, analyze, and test

Step 4 - seed the deterministic fixture twice

```
python -m flask demo seed  
python -m flask demo seed
```

- First run: expect 4 tasks and 143 inserted samples.
- Second run: expect 0 created tasks and 0 inserted samples. Say: 'This demonstrates idempotency.'

Step 5 - generate training-only EDA

```
python -m flask analysis eda --output reports/generated  
Start-Process reports/generated/eda.html
```

- Show the CPU and memory distributions, temporal lines, and request-versus-maximum scatter plots.
- Point out that the fixture is generated and that real findings require the prepared Google sample.

Step 6 - run the validation persistence baseline

```
python -m flask baseline run `--dataset-version synthetic-fixture`  
--output reports/generated/persistence_baseline.json  
Get-Content reports/generated/persistence_baseline.json
```

- Explain that validation metrics are stored in the database and JSON artifact.
- Do not add --include-test during development; that option is reserved for final frozen evaluation.

Step 7 - run automated verification

```
python -m pytest
```

- Expected current result: 13 passed.

DEDICATED CURRENT-STATE RUN SECTION

Person 4 - live demo: start the app and explain outputs

Step 8 - start Flask

```
python -m flask run
```

Step 9 - open these pages in order

Address	What to show	What to say
http://127.0.0.1:5000/	Milestone landing page	The UI honestly lists what is implemented and what is deferred.
http://127.0.0.1:5000/api/health	JSON health response	The Flask process can query its configured database.
http://127.0.0.1:5000/api/workloads/summary	Counts and average resource values	The API reads actual persisted workload rows, not hard-coded JSON.
.../summary?start=0&end=3600000000	Filtered summary	Trace-relative microsecond filters are validated server-side.

If something fails live

Keep the terminal visible and explain the error. Then show the already generated eda.html, persistence_baseline.json, and the latest pytest output. Do not substitute synthetic results for real-trace claims.

Close the demo with three statements

- The data path is reproducible and rejects corruption or invalid values.
- The database and API are operational, and duplicate ingestion is safe.
- The persistence baseline is the comparison point for the advanced models in the next milestone.

USE DURING EXPLANATION OR VIVA

Code responsibility reference - application and data

File or area	Responsibility	Called by / visible in demo
wsgi.py	Creates the Flask application object used by the development server.	python -m flask run
cloudalloc/__init__.py	Application factory: loads configuration, initializes database/migrations, registers API, dashboard, and CLI.	Every Flask command and request
cloudalloc/config.py	Reads secret, database URL, artifact directory, and SQLAlchemy settings.	DATABASE_URL environment variable
cloudalloc/extensions.py	Owns shared SQLAlchemy and Flask-Migrate objects without creating import cycles.	Application startup and migrations
cloudalloc/constants.py	Defines normalized schema columns, interval length, later model/policy names.	Preprocessing and baseline
cloudalloc/models.py	Defines tasks, samples, experiments, predictions, decisions, and metric tables plus indexes/constraints.	Migration, seed, baseline, API
cloudalloc/preprocessing.py	Streams Google JSONL/GZIP, normalizes fields, validates data, fills one gap, assigns splits, writes/verifies manifest.	flask data prepare / ingest
cloudalloc/ingestion.py	Creates missing tasks and inserts only unseen task-time samples.	flask data ingest and demo seed
cloudalloc/sample_data.py	Builds deterministic periodic fixture and its Parquet/manifest artifacts.	flask demo seed
scripts/extract_sample.sql	Optional BigQuery extraction for the same seven-day/hash/coverage design.	Not required for fixture demo

USE DURING EXPLANATION OR VIVA

Code responsibility reference - analysis, interface, and quality

File or area	Responsibility	Called by / visible in demo
cloudalloc/analysis.py	Queries stored samples and creates descriptive JSON plus six interactive Plotly views; defaults to train split.	flask analysis eda
cloudalloc/baseline.py	Builds exact t to t+1 persistence pairs, calculates four forecast metrics, stores experiment/predictions/metrics.	flask baseline run
cloudalloc/api.py	Implements database health and workload summary routes with time-filter validation.	/api/health and /api/workloads/summary
cloudalloc/dashboard.py	Serves the current milestone landing page.	GET /
cloudalloc/templates/index.html	Explains current capability and deferred work in the browser.	Landing page
cloudalloc/cli.py	Connects human-readable Flask commands to preparation, ingestion, EDA, baseline, and demo services.	All demo commands
migrations/versions/0001_initial_schema.py	Creates and reverses the initial relational schema reproducibly.	flask db upgrade / current
tests/test_preprocessing.py	Verifies hash, validation, gaps, splits, checksum, and no future-event join.	pytest
tests/test_ingestion_api.py	Verifies idempotency, health, summary, and invalid query responses.	pytest
tests/test_baseline.py	Verifies contiguous forecasts, metric correctness, and persistence of results.	pytest
tests/test_cli.py	Runs fixture seed and EDA through Flask's CLI runner.	pytest
docker-compose.yml / .env.example	Provides the intended PostgreSQL service and example runtime configuration.	PostgreSQL setup, not quick demo
pyproject.toml	Declares Python version, runtime libraries, development tests, and package metadata.	pip install -e .[dev]

SHARED CLOSING AND VIVA PREPARATION

Likely questions, limitations, and next 70%

Short answers to likely questions

Question	Recommended answer
Why not use the full Google trace?	It is roughly 2.4 TiB compressed. Deterministic sampling makes the experiment feasible and reproducible on student hardware.
Why is persistence necessary?	Many workloads are temporally stable. A complex model must beat a strong simple baseline, not only a weak random guess.
Why predict maximum rather than average?	Allocation must cover short peaks; average demand alone can hide under-provisioning risk.
Why is EDA train-only?	Inspecting test behavior can influence model choices. Keeping it unseen makes final evaluation more credible.
Why SQLite in the demo?	It minimizes operational risk while exercising the same ORM schema and services. PostgreSQL remains the documented deployment target.
Are the current metrics research results?	No. They are fixture-based software validation. Real claims begin only after official trace ingestion.
What proves there is no temporal leakage?	Chronological splits, latest-prior event joins, past-only gap fill, and exact five-minute forecast pairing are implemented and tested.

Remaining work

- 30-50%: construct lag/rolling features and train Ridge, random forest, and histogram gradient boosting.
- 50-70%: simulate static, reactive, predictive, and oracle allocations; calculate waste, deficit, utilization, churn, and confidence intervals.
- 70-85%: add experiment, comparison, and recommendation APIs plus the final analytics dashboard.
- 85-100%: execute the frozen test evaluation, robustness checks, final report, slides, and final demonstration.

Final team closing

At 30%, we have not proven that predictive allocation is better. We have built the reproducible data, database, analysis, baseline, API, and testing foundation needed to answer that question correctly in the remaining work.

ONE MINUTE BEFORE PRESENTING

Final checklist and references

Team checklist

Owner	Ready when...
Person 1	Can state problem, gap, four research questions, dataset choice, and non-overclaim boundary.
Person 2	Can trace one row from raw JSON to Parquet manifest to relational sample and explain idempotency.
Person 3	Can define persistence and all four metrics, explain the two API routes, and summarize test coverage.
Person 4	Has run every demo command once, has backup artifacts open, and can explain synthetic versus real data.
All	Use normalized capacity units; never call them vCPU or GB. Say 30% foundation, not completed predictive allocator.

Primary project references

- [Google Cluster Trace 2019 documentation and access notes](#)
- [Tirmazi et al. \(2020\), Borg: the Next Generation](#)
- [Cortez et al. \(2017\), Resource Central](#)
- [Hadary et al. \(2020\), Protean](#)
- Full 13-study matrix: docs/literature_matrix.md
- Research design: docs/research_design.md
- 30% progress report: reports/30_percent_progress.md

Recommended final sentence

The current milestone makes later comparisons trustworthy because every future model and allocation policy will use the same validated records, chronological boundaries, database, and evaluation pipeline.